# Synchronous Serial Interfaces: SPI and $I^2C$

*Introduction to ARM Cortex-M Microprocessors* — Chapter 13

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

# Table of Contents


1. Why This Matters
2. The SPI Bus
3. Clock Polarity and Phase
4. Clock Frequency
5. SPI Registers
6. Configuring an SPI Master
7. A Register Transaction
8. Application: Reading an Accelerometer
9. $I^2C$: Two Wires, Many Devices
10. Comparing the Serial Protocols
11. Summary

# Why This Chapter Matters

UART has no clock wire, so both ends must agree on the baud rate. Adding a clock wire removes that need and the speed limit. SPI is fast and simple, but costs one pin per device. $I^2C$ is slower, yet it connects dozens of devices on two wires. You will read a real sensor and turn its raw values into a physical measurement.

## After this chapter, you can:

1. Name the four SPI signals; explain full-duplex exchange.
2. Choose CPOL and CPHA; calculate $f_{SCK}$.
3. Configure an SPI master and do a register transaction.
4. Describe the $I^2C$ phases; explain open-drain and pull-ups.
5. Configure an $I^2C$ master and read a slave register.
6. Choose between SPI and $I^2C$, and justify it.
7. Convert a real sensor's output to engineering units.

© Muním Zabidi

By the end of this chapter you will know how this call asks a chip on the board who it is, using four wires:

```
uint8_t id = sensor_read_reg(0x00);    // DEVID register
// the ADXL345 answers 0xE5
```

Questions this code raises, and where this chapter answers them:

| Question raised | Answered in |
|---|---|
| Why four wires? | The SPI bus |
| Why does the byte come back as we send? | Full duplex by shift register |
| What if the mode is wrong? | Clock polarity and phase |
| What if I need many sensors on two wires? | $I^2C$ |

# Table of Contents


1 Why This Matters

2 **The SPI Bus**

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

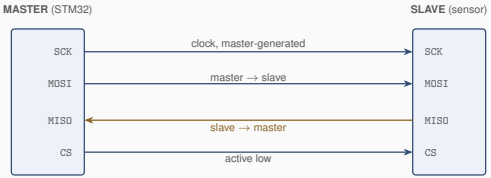
8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# Four Wires, Full Duplex

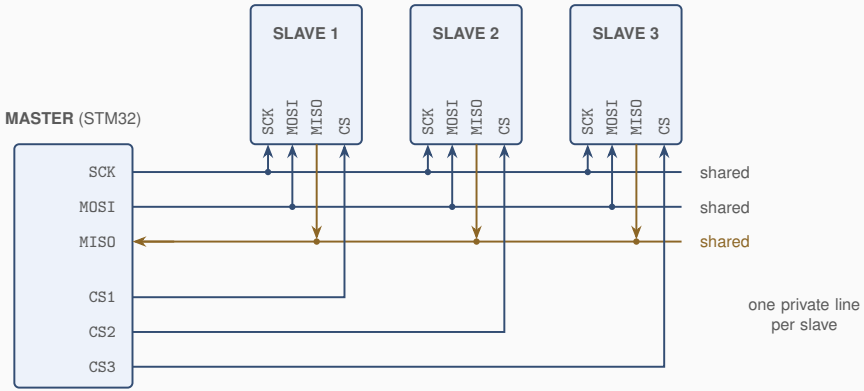



| Signal | Direction |
|---|---|
| SCK | Master → slave |
| MOSI | Master → slave |
| MISO | Slave → master |
| CS | Master → slave, per device |

## A transfer is an *exchange*, not a send

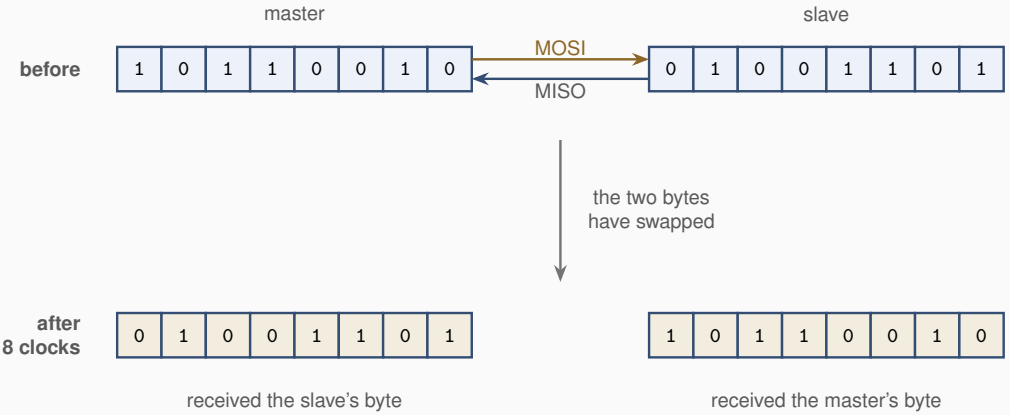
Master and slave are one circular shift register. Every clock edge shifts a bit out of each and into the other — so sending a byte *always* receives one at the same time. To read a device you must still write something (often a dummy byte) to generate the clock.

# More Than One Slave




`SCK`, `MOSI`, and `MISO` are shared by every device; only chip select is private. Adding a slave costs exactly one pin.

# Full Duplex by Shift Register




SPI is an exchange, not a send or a receive. The two shift registers form one ring, so eight clock pulses trade their contents.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 **Clock Polarity and Phase**

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

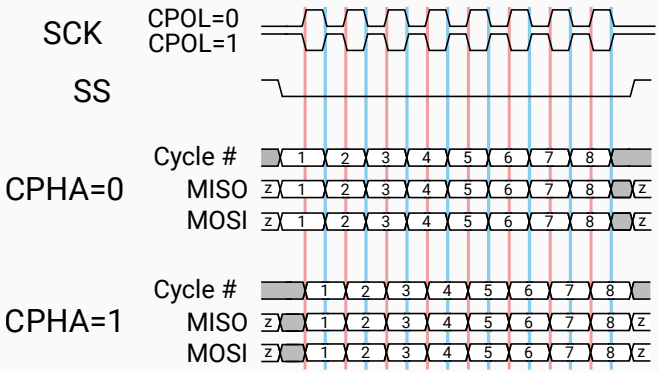
8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# Clock Polarity and Phase




**CPOL** sets the idle level of the clock; **CPHA** sets which edge samples the data. Four combinations, called modes 0 to 3 — and the datasheet tells you which one your device needs.

# The Four SPI Modes

| Mode | CPOL | CPHA | SCK idle | Data sampled on |
|---|---|---|---|---|
| 0 | 0 | 0 | Low | Rising edge (first) |
| 1 | 0 | 1 | Low | Falling edge (second) |
| 2 | 1 | 0 | High | Falling edge (first) |
| 3 | 1 | 1 | High | Rising edge (second) |

**CPOL** only inverts the clock waveform. **CPHA** decides which edge samples the data. With CPHA $= 0$ the data must already be stable when the clock first moves.

## A wrong mode looks like a wiring fault

Data arrives shifted by one bit, or read at the wrong instant. It looks plausible but is wrong. Mode 0 is the most common. The slave's datasheet states the mode.

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 **Clock Frequency**

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# Clock Frequency

The master divides its peripheral bus clock by a power of two, chosen by the three `BR` bits:

$$f_{SCK} = \frac{f_{PCLK}}{2^{(BR+1)}}$$

| BR | Divide by | SCK from 16 MHz |
|---|---|---|
| `000` | 2 | 8 MHz |
| `011` | 16 | 1 MHz |
| `111` | 256 | 62.5 kHz |

Choose the highest speed the slave supports *and* the wiring allows. Long or unshielded wires need a slower clock. A breadboard rarely tolerates more than a few megahertz.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 **SPI Registers**

6 Configuring an SPI Master

7 A Register Transaction

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# SPI Registers

| Register | Bit | Name | Function |
|---|---|---|---|
| SPI_CR1 | 9 | SSM | Software slave management |
| | 8 | SSI | Internal slave select. Set with SSM when CS is a GPIO |
| | 7 | LSBFIRST | Bit order; normally 0 (MSB first) |
| | 6 | SPE | SPI enable |
| | 5:3 | BR | Baud rate prescaler |
| | 2 | MSTR | Master mode |
| | 1 | CPOL | Idle level of SCK |
| | 0 | CPHA | Which edge samples |
| SPI_SR | 7 | BSY | Transfer in progress |
| | 1 | TXE | Transmit buffer empty |
| | 0 | RXNE | Receive buffer not empty |
| SPI_DR | 15:0 | | Writing starts a transfer; reading gets the byte received |

If the master drives CS as a GPIO, set both SSM and SSI. Otherwise the peripheral thinks it lost arbitration and disables itself.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 **Configuring an SPI Master**

7 A Register Transaction

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# Why SPI2, Not SPI1 A pin-assignment decision

| Pin | Use |
|---|---|
| PB13 | SCK (AF5) |
| PB14 | MISO (AF5) |
| PB15 | MOSI (AF5) |
| PB12 | CS, a plain GPIO output |

## SPI1 would break the LED

SPI1's default SCK is PA5, the user LED used all semester. SPI2 keeps the LED free, so the new sensor joins the running system.

Details that are easy to get wrong: SPI2 is on APB1 (`RCC->APB1ENR`, bit 14). `PB12`–`PB15` are in `AFR[1]`, indexed as $(\text{pin} - 8) \times 4$. Checking for collisions like this before choosing pins is ordinary design work. Chapter 14 treats it that way.

# Configuring an SPI Master

```
RCC->APB1ENR |= (1U « 14); // SPI2

SPI2->CR1 = (1U « 2)       // MSTR
          | (3U « 3)       // BR: /16
          | (1U « 9)       // SSM
          | (1U « 8);      // SSI

SPI2->CR1 |= (1U « 6);     // SPE
```

$$f_{SCK} = \frac{f_{PCLK}}{2^{(BR+1)}}$$

With BR $= 3$ and a 16 MHz peripheral clock, $f_{SCK} = 1$ MHz.

## Why this course uses SPI2, not SPI1

SPI1's default SCK is PA5 — the user LED. Using SPI1 would silently break every blink program in the book. SPI2 sits on **APB1** (not APB2), and its pins PB13–PB15 live in AFR[1], indexed as $(pin - 8) \times 4$.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 **A Register Transaction**

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# A Register Transaction

```
uint8_t spi_transfer(uint8_t out)
{
    while (!(SPI2->SR & (1U << 1))) ;    // TXE -- ready to send
    *(volatile uint8_t *)&SPI2->DR = out;
    while (!(SPI2->SR & (1U << 0))) ;    // RXNE -- a byte came back
    return *(volatile uint8_t *)&SPI2->DR;
}

uint8_t read_register(uint8_t addr)
{
    cs_low();
    spi_transfer(addr | 0x80);       // bit 7 set = read
    uint8_t v = spi_transfer(0x00);  // dummy byte to clock the reply out
    cs_high();
    return v;
}
```

The dummy byte is the point: you cannot receive without transmitting, because the master supplies the clock.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

8 **Application: Reading an Accelerometer**

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 Summary

# Application: Reading an Accelerometer ADXL345 over SPI

```
#define REG_DEVID        0x00
#define REG_POWER_CTL    0x2D
#define REG_DATA_FORMAT  0x31
#define REG_DATAX0       0x32

void accel_init(void) {
    spi_init();
    sensor_write_reg(REG_DATA_FORMAT, 0x00); // +/-2g
    sensor_write_reg(REG_POWER_CTL,   0x08); // start
}

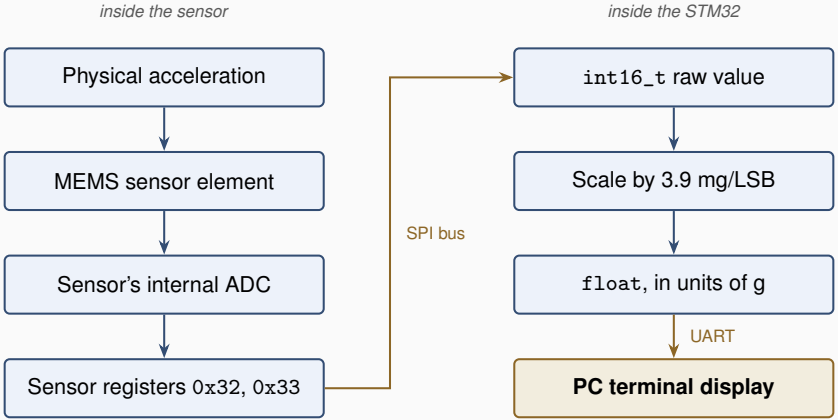
int16_t accel_read_axis(uint8_t reg_low) {
    uint8_t lo = sensor_read_reg(reg_low);
    uint8_t hi = sensor_read_reg(reg_low + 1);
    return (int16_t)((uint16_t)hi << 8 | lo);
}

float accel_to_g(int16_t raw) {
    return (float)raw * 0.0039f;   // 3.9 mg/LSB
}
```

**The cast to `int16_t` is essential.** Acceleration is signed. Rebuilt as an unsigned type, every negative reading becomes a large positive one.

**Read the ID first.** The ADXL345 returns `0xE5`. If you read `0x00` or `0xFF`, the bus is not working, and no more code will help.

# The Complete Measurement Chain




From a physical acceleration to a number on a screen. Every stage is a topic from an earlier chapter.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

8 Application: Reading an Accelerometer

9 **$I^2C$: Two Wires, Many Devices**

10 Comparing the Serial Protocols

11 Summary

# Two Wires, Many Devices

- `SCL` clock and `SDA` data — two wires total, regardless of device count
- Each device has a 7-bit address, so no chip-select lines
- Both lines are open-drain, with external pull-ups

### Why open-drain is mandatory

Many devices share both wires. If two drove push-pull outputs in opposite directions the result would be a short circuit.

Open-drain means a device can only *pull low* or *release* — so the pull-up defines the idle state and no contention is possible.

That is Chapter 8's open-drain output, now with a reason. The pull-up value follows from bus capacitance: too large and the rising edge is too slow; too small and it wastes current.

© Muním Zabidi

# The Pull-Up Value Is a Compromise

The bus line and its devices present a capacitance $C_b$. The pull-up must charge it within the rise time the standard allows:

$$t_r \approx 0.85\, R_p C_b$$

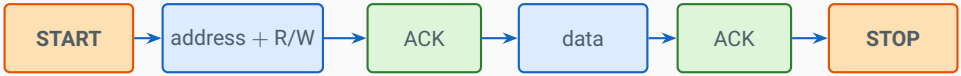
A smaller resistor gives faster edges, but draws more current whenever a device pulls the line low.

With $C_b = 100$ pF, a 4.7 kΩ pull-up gives about 400 ns. That suits 100 kHz, but is marginal at 400 kHz, where 1 kΩ is better.

## The most common I²C fault

Missing pull-ups. Symptom: SDA and SCL sit near 0 V and every transfer fails at once.

# A Transaction

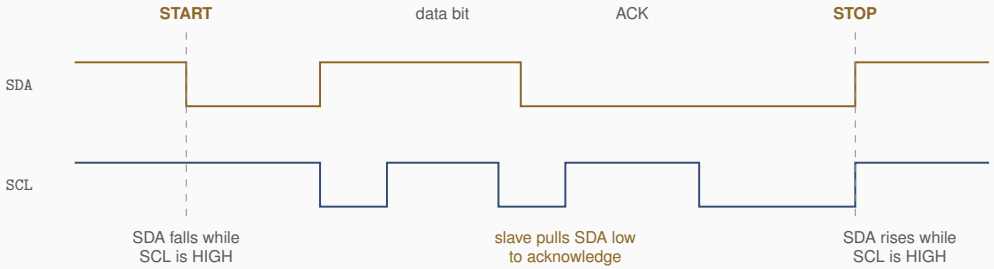



| Phase | What happens |
|---|---|
| **START** | SDA falls while SCL is high — the one pattern that is otherwise illegal |
| **Address** | 7 bits plus a read/write bit. Every device compares it against its own |
| **ACK** | The addressed device pulls SDA low. Silence means nobody answered |
| **STOP** | SDA rises while SCL is high |

A missing ACK is the most common $I^2C$ fault — and it usually means a wrong address or missing pull-ups.

© Muním Zabidi

# The Framing Conditions, on the Wire




SDA may only change while SCL is LOW, so the two transitions that occur while SCL is HIGH are unambiguous markers for the start and end of a transfer.

Each device answers to a 7-bit address: 128 possibilities, a few reserved. The manufacturer fixes it, often with one or two pins to select between variants. That is how two identical sensors share one bus.

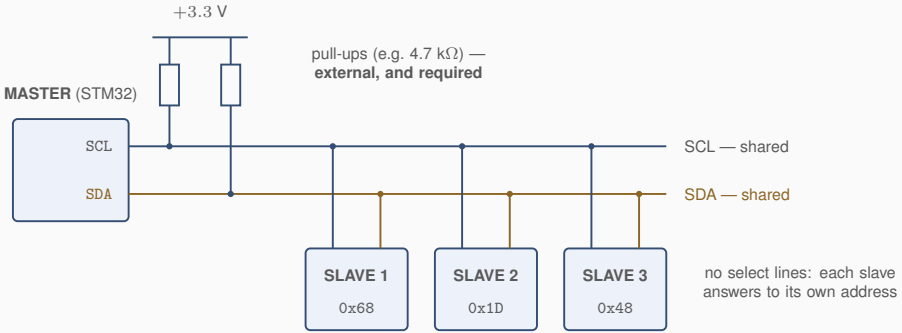
| Datasheet says | On the wire, write | On the wire, read |
|---|---|---|
| `0x68` (7-bit) | `0xD0` | `0xD1` |

## A device that never acknowledges

The address goes in the upper seven bits of the first byte, with R/$\overline{W}$ in bit 0. A device that never ACKs is usually being addressed with the 7-bit value shifted the wrong way. The STM32 register expects the already-shifted 8-bit form.

© Muním Zabidi

# Many Devices on Two Wires




Compare this with SPI's three private chip-select lines: here the pin count stays at two, no matter how many devices hang off the bus — selection has moved into software.

# I²C Registers

| Register | Bit | Name | Function |
|---|---|---|---|
| I2C_CR1 | 15 | SWRST | Software reset |
| | 10 | ACK | Acknowledge enable; clear to NACK the next byte |
| | 9 | STOP | Generate a stop condition |
| | 8 | START | Generate a start (or repeated start) |
| | 0 | PE | Peripheral enable |
| I2C_CR2 | 5:0 | FREQ | PCLK1 in MHz, used to time the bus |
| I2C_CCR | 11:0 | | $f_{PCLK1}/(2 \times f_{SCL})$ in standard mode |
| I2C_TRISE | 5:0 | | Maximum SCL rise time, in FREQ units |
| I2C_SR1 | 7 | TXE | Data register empty |
| | 6 | RXNE | A byte has arrived |
| | 2 | BTF | Byte transfer finished |
| | 1 | ADDR | Address sent and acked; cleared by reading SR1 then SR2 |
| | 0 | SB | Start condition generated |
| I2C_SR2 | 1 | BUSY | Communication in progress |
| I2C_DR | 7:0 | | Carries the address byte and the data |

The status bits are split across two registers, SR1 and SR2. This matters for ADDR.

```c
void i2c_init(void) {
    RCC->AHB1ENR |= (1U << 1);   // GPIOB
    RCC->APB1ENR |= (1U << 21);  // I2C1

    GPIOB->MODER &= ~((3U<<(8*2)) | (3U<<(9*2)));
    GPIOB->MODER |=  ((2U<<(8*2)) | (2U<<(9*2)));

    // OPEN-DRAIN. Without this the bus cannot work.
    GPIOB->OTYPER |= (1U << 8) | (1U << 9);

    // AF4 = I2C1; pins 8-15 are in AFR[1]
    GPIOB->AFR[1] &= ~((0xFU << 0) | (0xFU << 4));
    GPIOB->AFR[1] |=  ((4U << 0) | (4U << 4));

    I2C1->CR1 |= (1U << 15);     // SWRST
    I2C1->CR1 &= ~(1U << 15);    // release
    I2C1->CR2  = 16;             // FREQ: PCLK1 in MHz
    I2C1->CCR  = 80;             // 16 MHz / (2 x 100 kHz)
    I2C1->TRISE = 17;            // FREQ + 1
    I2C1->CR1 |= (1U << 0);      // PE
}
```

## The forgotten step

Both pins must be open-drain alternate-function outputs. This step has no counterpart in SPI, and it is the one most often forgotten.

$$\text{CCR} = \frac{f_{\text{PCLK1}}}{2f_{\text{SCL}}} = 80$$

TRISE = FREQ + 1 in standard mode.

A new I²C master is wired up and configured. Every transfer fails at once. On the scope, SDA and SCL sit near **0 V**.

**Your prediction**

- Who should pull a released line high?
- Which GPIO setting is easy to forget?
- Which is the most common I²C fault?

A new I²C master is wired up and configured. Every transfer fails at once. On the scope, SDA and SCL sit near **0 V**.

**Your prediction**

- Who should pull a released line high?
- Which GPIO setting is easy to forget?
- Which is the most common I²C fault?

## The answer

Open-drain pins can only pull low. External pull-up resistors bring a released line high. Two causes: the pull-ups are missing, or the pins were not set open-drain (`OTYPER`). Missing pull-ups is the single most common fault.

© Munim Zabidi

# A Register Read Over I²C (1) Phases 1 and 2

```
// addr: the 7-bit address ALREADY shifted left by 1
uint8_t i2c_read_reg(uint8_t addr, uint8_t reg) {
    while (I2C1->SR2 & (1U << 1)) { }     // bus free?

    // --- Phase 1: START, address for WRITE ---
    I2C1->CR1 |= (1U << 8);               // START
    while (!(I2C1->SR1 & (1U << 0))) { }  // SB: start sent
    I2C1->DR = addr & ~1U;                // address + write bit
    while (!(I2C1->SR1 & (1U << 1))) { }  // ADDR: address acked
    (void)I2C1->SR2;                      // SR1 then SR2 clears ADDR

    // --- Phase 2: send the register number ---
    while (!(I2C1->SR1 & (1U << 7))) { }  // TXE: DR empty
    I2C1->DR = reg;
    while (!(I2C1->SR1 & (1U << 2))) { }  // BTF: byte done
```

Each step waits on a status flag: the busy-wait pattern, applied to a peripheral whose every phase must be confirmed.

# A Register Read Over I²C (2) Phases 3 and 4

```
    // --- Phase 3: repeated START, address for READ ---
    I2C1->CR1 |= (1U << 8);                 // repeated START
    while (!(I2C1->SR1 & (1U << 0))) { }    // SB
    I2C1->DR = addr | 1U;                   // address + read bit
    while (!(I2C1->SR1 & (1U << 1))) { }    // ADDR
    I2C1->CR1 &= ~(1U << 10);               // ACK = 0: NACK
    (void)I2C1->SR2;                        // clear ADDR

    // --- Phase 4: receive one byte, then STOP ---
    I2C1->CR1 |= (1U << 9);                 // STOP
    while (!(I2C1->SR1 & (1U << 6))) { }    // RXNE
    return (uint8_t)I2C1->DR;
}
```

## Two STM32 traps

`ADDR` is cleared by reading `SR1` *then* `SR2`. The `(void)I2C1->SR2` is not dead code: delete it and the bus hangs.

For a one-byte read, clear `ACK` *before* clearing `ADDR`.

**Add a timeout** to every loop. A missing pull-up or an unpowered slave means a flag never sets, and the function never returns.

# Choosing Between Them

| | SPI | $I^2C$ |
|---|---|---|
| Wires | 4, plus one CS per device | 2, regardless of count |
| Speed | Tens of MHz | 100 kHz to 1 MHz |
| Duplex | Full | Half |
| Addressing | A chip-select line | A 7-bit address |
| Acknowledge | None — you cannot tell if it worked | ACK per byte |
| Best for | Displays, flash, fast ADCs | Many slow sensors, EEPROM |

## The trade in one sentence

SPI buys speed with pins; $I^2C$ buys pins with speed — and throws in an acknowledgement, so you know whether the device is actually there.

© Muním Zabidi

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 **Comparing the Serial Protocols**

11 Summary

# Comparing the Serial Protocols

| Property | UART | SPI | $I^2C$ |
|---|---|---|---|
| Clock | None (async) | Shared SCK | Shared SCL |
| Wires | 2 | 4 (+1 per slave) | 2 |
| Speed | to ∼1 Mbps | tens of Mbps | 0.1–3.4 Mbps |
| Duplex | Full | Full | Half |
| Devices | 2 | Many (one CS each) | Many (addressed) |
| Addressing | None | Chip select | 7-bit address |
| Typical use | PC terminal, GPS | Sensors, displays, SD | Small sensors, EEPROM |

## A NACK is diagnostic information

$I^2C$ is the only one with addressing in the protocol, so it is the only one where a device can refuse to answer. An unpowered SPI slave returns `0x00` or `0xFF`, which looks like real data. An absent $I^2C$ device announces its absence on the first byte.

# Table of Contents


1 Why This Matters

2 The SPI Bus

3 Clock Polarity and Phase

4 Clock Frequency

5 SPI Registers

6 Configuring an SPI Master

7 A Register Transaction

8 Application: Reading an Accelerometer

9 $I^2C$: Two Wires, Many Devices

10 Comparing the Serial Protocols

11 **Summary**

# An SPI Byte at Five Levels

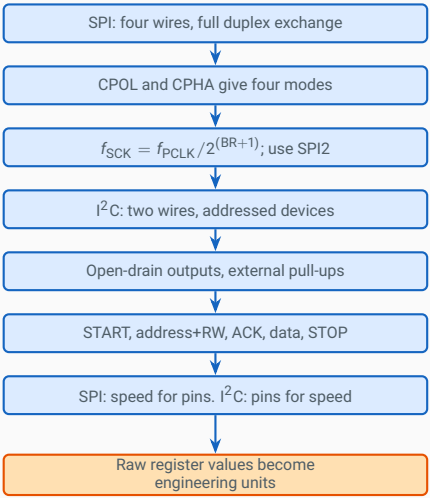
| Level | Representation |
|---|---|
| Physical | Eight clock pulses on PB13; data levels on PB15 and PB14 |
| Hardware | Two shift registers exchanging contents in a ring |
| Register | `SPI2->DR` written, then read |
| C | `value = spi_transfer(0x00);` |
| Engineering | One axis of acceleration, in units of g |

**You have now met every peripheral this book teaches**

Digital pins, interrupts, timers, analog input, and three serial buses: one asynchronous and two synchronous. Chapter 14 asks you to use them all at once, with no OS to arbitrate.

© Muním Zabidi

# Summary




**Terminology covered**

- MOSI, MISO, SCK, CS
- CPOL, CPHA
- Open-drain, pull-up
- START, ACK, STOP

**Next: Chapter 14**

*Designing, Building and Verifying*

Every peripheral so far, in one program, at once.

© Munim Zabidi